

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Алгоритмы ориентации в пространстве

Отчёт по лабораторной работе № 3

Работу
выполнил:
Сухов Н. М.
Группа: R3335
Преподаватель:
Каплин А. В.

Санкт-Петербург
2026

Содержание

1. Цель работы	3
2. Описание сцены в CoppeliaSim	3
3. Настройка камеры и калибровка изображения	3
4. Алгоритм обработки изображения	4
5. Вычисление расстояния и угла до линии	5
6. План эксперимента	6
7. Результаты эксперимента	6
8. Графики результатов	7
9. Анализ результатов	10
10. Вывод	10
11. Исходный код	10

1. Цель работы

Цель работы заключается в создании эталонной линии в среде CoppeliaSim, извлечении этой линии из данных сенсора, оценке относительной позы робота по расстоянию и углу до линии, а также анализе ошибок измерения.

В работе использовалась модель робота Pioneer3DX, объект-линия на поверхности пола и vision sensor, установленный на роботе. Обработка изображения выполнялась во внешнем Python-скрипте с использованием библиотек numpy, opencv-python, matplotlib и coppeliasim-zmqremoteapi-client.

2. Описание сцены в CoppeliaSim

В среде CoppeliaSim была создана сцена, включающая плоскость пола, мобильного робота Pioneer3DX, эталонную линию и vision sensor. Линия была задана в виде тонкого кубоида, расположенного на поверхности пола. Для повышения контраста линия была сделана тёмной относительно светлой поверхности пола.

Глобальная линия была расположена вдоль оси X мировой системы координат. Центр линии был принят равным

$$P_c = (0; 0)$$

Длина линии была принята равной

$$L = 5 \text{ м}$$

Тогда координаты концов линии в плоскости XY имеют вид

$$P_1 = (-2.5; 0), \quad P_2 = (2.5; 0)$$

Глобальное уравнение линии имеет вид

$$y = 0$$

В общем виде уравнение прямой записывается как

$$ax + by + c = 0$$

Для данной линии коэффициенты равны

$$a = 0, \quad b = 1, \quad c = 0$$

На рисунке [2.1](#) показана подготовленная сцена в CoppeliaSim.

3. Настройка камеры и калибровка изображения

В качестве сенсора использовался vision sensor /Pioneer3DX/LineCamera. Камера была закреплена на роботе и направлена на поверхность пола. Такой вариант установки соответствует схеме top-down и упрощает перевод пиксельных координат в метрические, так как плоскость изображения приближённо соответствует плоскости пола.

В эксперименте использовались следующие параметры камеры:

- угол обзора камеры: 135° ;

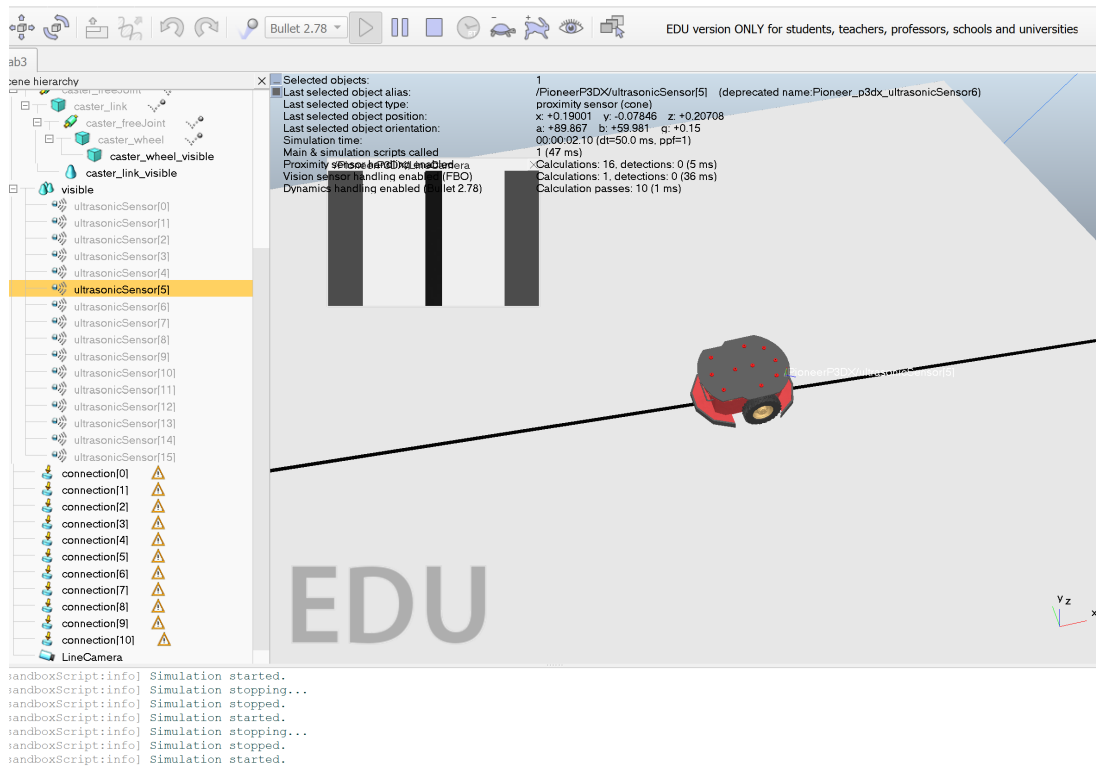


Рисунок 2.1. Сцена CoppeliaSim с роботом PioneerP3DX, эталонной линией и vision sensor

- высота камеры над плоскостью пола: $h = 0.03879$ м;
- фактическое разрешение изображения считывалось из CoppeliaSim автоматически;
- в скрипте использовался масштаб ширины видимой области $W = 0.187$ м.

Приближённая ширина области обзора определялась по формуле

$$W = 2h \tan \frac{\alpha}{2}$$

Подстановка параметров камеры даёт

$$W = 2 \cdot 0.03879 \cdot \tan 67.5^\circ \approx 0.187 \text{ м}$$

Масштаб пикселя по горизонтали вычислялся как

$$m_{px} = \frac{W}{N_x}$$

где N_x - ширина изображения в пикселях.

4. Алгоритм обработки изображения

Обработка изображения выполнялась во внешнем Python-скрипте. Общая последовательность обработки следующая:

1. подключение к CoppeliaSim через ZeroMQ Remote API;
2. получение кадров с объекта /PioneerP3DX/LineCamera;

3. преобразование RGB-изображения в формат BGR для обработки в OpenCV;
4. перевод изображения в оттенки серого;
5. сглаживание изображения фильтром Гаусса;
6. выделение границ алгоритмом Canny;
7. поиск отрезков линии методом вероятностного преобразования Хафа HoughLinesP;
8. при недостаточном числе найденных точек используется пороговая маска тёмных пикселей;
9. аппроксимация общей прямой по найденным точкам функцией `cv2.fitLine`;
10. вычисление расстояния и угла линии в системе координат робота;
11. сохранение исходных кадров, обработанных кадров, таблицы измерений и графиков ошибок.

Пример результата обработки изображения показан на рисунке 4.1. Зелёной линией показана аппроксимированная ось найденной линии, синей точкой отмечен центр изображения.

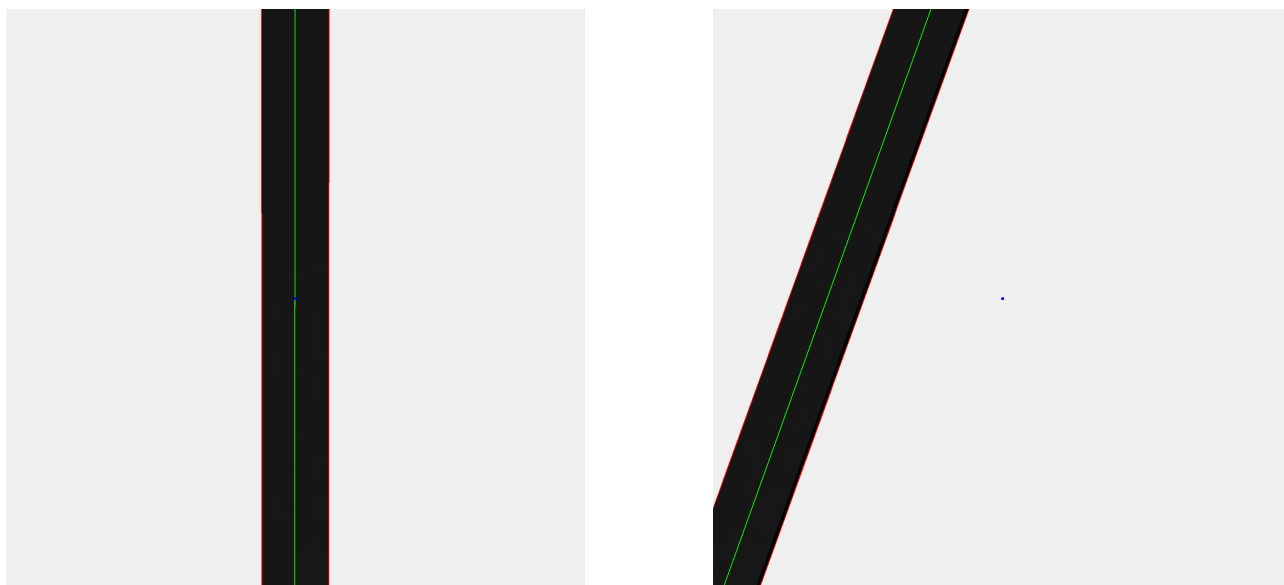


Рисунок 4.1. Примеры кадров после обработки и аппроксимации линии

5. Вычисление расстояния и угла до линии

После перевода пиксельных координат в метрические координаты точки линии аппроксимированы прямой. Пусть линия в системе координат робота задана уравнением

$$ax + by + c = 0$$

Расстояние от начала координат робота до линии вычислялось по формуле

$$d = \frac{c}{\sqrt{a^2 + b^2}}$$

В отчёте использовалось абсолютное значение расстояния

$$d_{abs} = |d|$$

Угол линии определялся по направляющему вектору прямой $\vec{v} = (v_x, v_y)$

$$\varphi = \text{atan2}(v_y, v_x)$$

Так как прямая не имеет выделенного направления, углы φ и $\varphi + 180^\circ$ описывают одну и ту же линию. Поэтому ошибка угла вычислялась с учётом периодичности 180°

$$\Delta\varphi = ((\varphi_{meas} - \varphi_{true} + 90^\circ) \bmod 180^\circ) - 90^\circ$$

6. План эксперимента

Для проверки алгоритма была проведена серия из девяти экспериментов. В первой группе робот располагался без поворота относительно линии на разных расстояниях: 0, 0.02, 0.04, 0.06 и 0.08 м. Во второй группе расстояние было фиксированным и равным 0.05 м, а робот поворачивался относительно линии на углы $+10^\circ$, -10° , $+20^\circ$ и -20° .

Для каждого опыта сохранялись:

- исходное изображение с камеры;
- обработанное изображение с найденной линией;
- истинное расстояние до линии;
- измеренное расстояние до линии;
- ошибка расстояния;
- истинный угол линии;
- измеренный угол линии;
- ошибка угла;
- признак успешной детекции.

7. Результаты эксперимента

Результаты измерений приведены в таблице ???. Ошибка расстояния представлена в сантиметрах, ошибка угла - в градусах.

Таблица 7.1

Результаты измерения расстояния и угла до линии

№	d_{true} , М	d_{meas} , М	Δd , см	φ_{true} , град	φ_{meas} , град	$\Delta\varphi$, град	Детекция
1	0.000	0.000	0.01	-90.00	89.99	-0.01	1
2	0.020	0.026	0.58	-90.00	89.02	-0.98	1
3	0.040	0.043	0.29	-90.00	-89.91	0.09	1
4	0.060	0.066	0.65	-90.00	-89.73	0.27	1
5	0.080	0.084	0.35	-90.00	-89.92	0.08	1
6	0.050	0.047	-0.34	80.00	79.15	-0.85	1
7	0.050	0.053	0.34	-80.00	-79.70	0.30	1
8	0.050	0.053	0.32	70.00	70.26	0.26	1
9	0.050	0.055	0.51	-70.00	-71.23	-1.23	1

По результатам эксперимента доля успешных детекций составила

$$P_{det} = 1.000$$

Максимальная абсолютная ошибка расстояния составила

$$\Delta d_{max} = 0.00647 \text{ м}$$

Средняя абсолютная ошибка расстояния составила

$$\overline{\Delta d} = 0.00378 \text{ м}$$

Максимальная абсолютная ошибка угла составила

$$\Delta \varphi_{max} = 1.23^\circ$$

Средняя абсолютная ошибка угла составила

$$\overline{\Delta \varphi} = 0.45^\circ$$

8. Графики результатов

На рисунке 8.1 приведено сравнение истинного и измеренного расстояния до линии. Видно, что после калибровки масштаба измеренные значения близки к истинным.

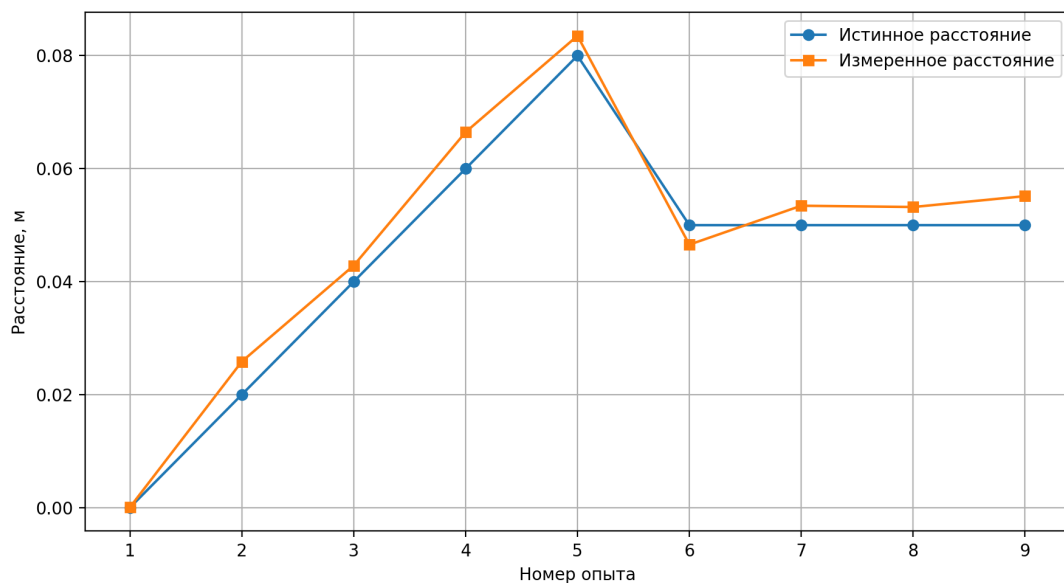


Рисунок 8.1. Сравнение истинного и измеренного расстояния до линии

На рисунке 8.2 показана ошибка измерения расстояния. Ошибка находится на уровне нескольких миллиметров.

На рисунке 8.3 приведено сравнение истинного и измеренного угла линии. При анализе графика необходимо учитывать, что направление прямой определено с точностью до 180° , поэтому значения -90° и $+90^\circ$ соответствуют одной и той же прямой.

На рисунке 8.4 показана ошибка измерения угла с учётом периодичности прямой 180° .

На рисунке 8.5 приведена диаграмма успешности детекции. Во всех проведённых опытах линия была обнаружена.

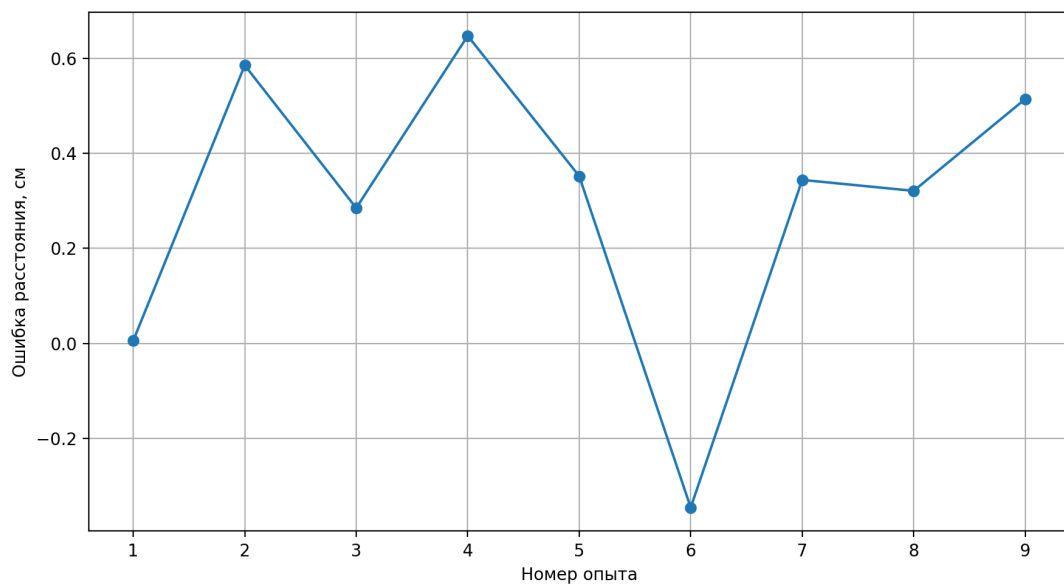


Рисунок 8.2. Ошибка измерения расстояния до линии

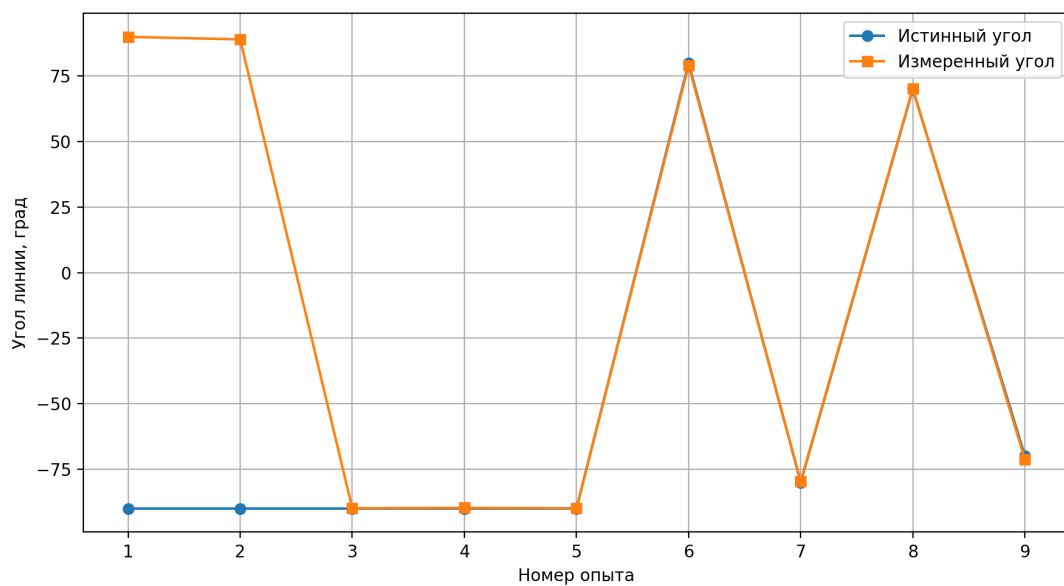


Рисунок 8.3. Сравнение истинного и измеренного угла линии

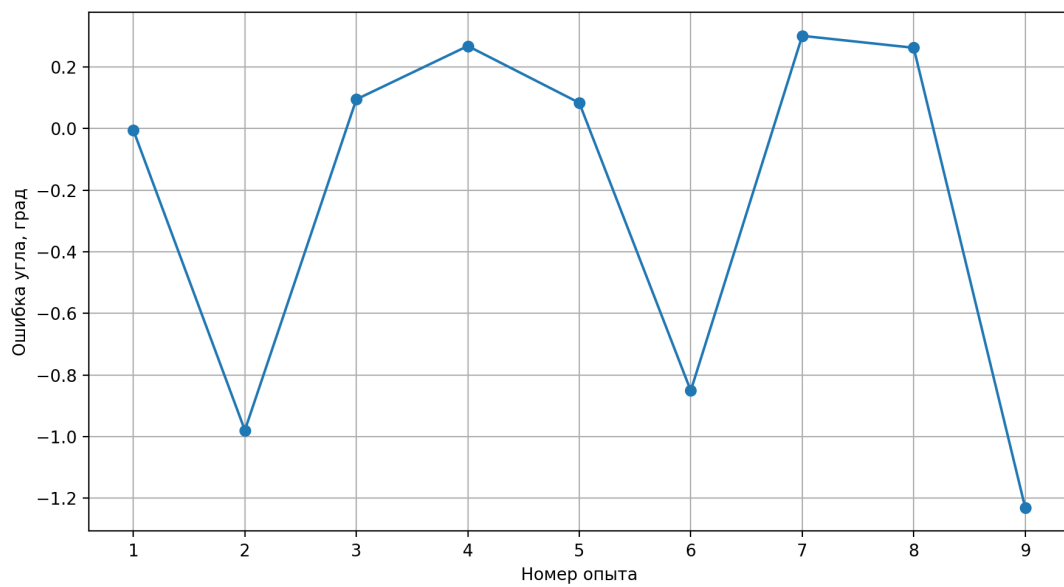


Рисунок 8.4. Ошибка измерения угла линии

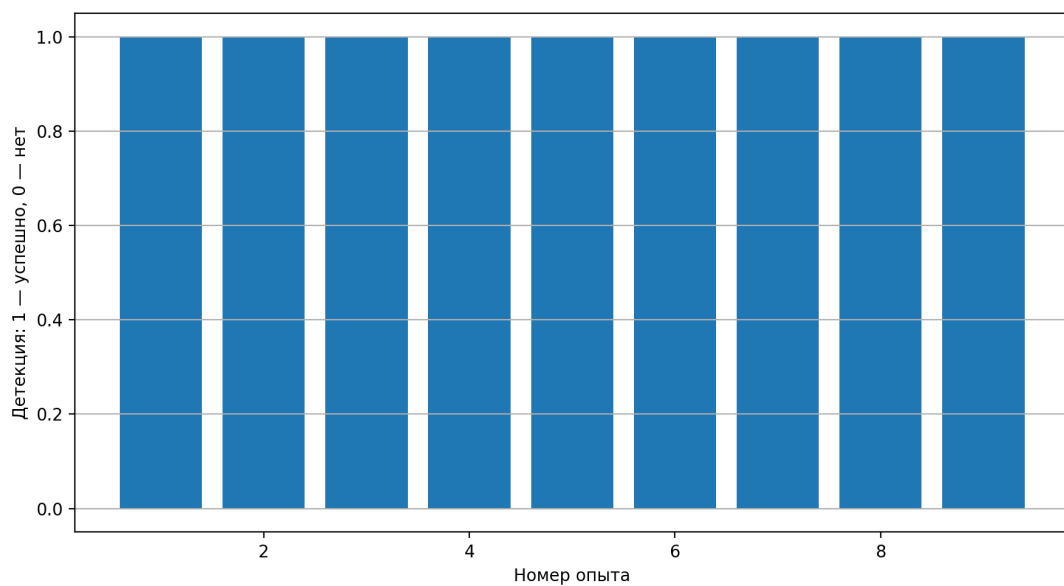


Рисунок 8.5. Доля успешных детекций линии по опытам

9. Анализ результатов

Полученные результаты показывают, что выбранный способ извлечения линии из изображения позволяет устойчиво находить эталонную линию в пределах поля зрения камеры. Во всех девяти опытах линия была обнаружена, что соответствует доле успешной детекции 1.000.

Основная погрешность расстояния связана с приближённой калибровкой масштаба изображения, конечной шириной линии, дискретностью пикселей и перспективными искажениями камеры. При этом максимальная ошибка расстояния не превысила 0.00647 м, что является приемлемым результатом для выбранной конфигурации сцены.

Ошибка определения угла оказалась небольшой: максимальное значение составило около 1.23° . На графике истинного и измеренного угла видны переходы между -90° и $+90^\circ$, однако они не являются ошибкой детекции, так как направление прямой задаётся с периодом 180° .

10. Вывод

В ходе лабораторной работы была создана сцена Coppeliasim с мобильным роботом Pioneer3DX и эталонной линией на поверхности пола. Был настроен vision sensor, подключён внешний Python-скрипт через ZeroMQ Remote API и реализован алгоритм извлечения линии из изображения с использованием OpenCV.

Выполнена калибровка масштаба изображения, позволяющая переводить пиксельные координаты в метрические координаты. Для серии тестовых положений робота были определены расстояние и угол до линии, построены графики ошибок и рассчитана доля успешных детекций.

В результате линия была успешно обнаружена во всех проведённых опытах. Максимальная абсолютная ошибка расстояния составила 0.00647 м, максимальная абсолютная ошибка угла составила 1.23° . Полученные результаты подтверждают работоспособность реализованного алгоритма локализации относительно линии.

11. Исходный код

Listing 1: Код программы для извлечения линии и оценки относительной позы робота

```
1 import csv
2 import math
3 import os
4 import time
5 from dataclasses import dataclass
6 from typing import List, Optional, Tuple
7
8 import cv2
9 import matplotlib.pyplot as plt
10 import numpy as np
11
12 try:
13     from coppeliasim_zmqremoteapi_client import RemoteAPIClient
14 except ImportError:
15     RemoteAPIClient = None
16
17
18 OUT_DIR = os.path.dirname(os.path.abspath(__file__))
```

```

19 RAW_DIR = os.path.join(OUT_DIR, "img", "raw")
20 PROC_DIR = os.path.join(OUT_DIR, "img", "processed")
21 RESULTS_DIR = os.path.join(OUT_DIR, "results")
22
23 ROBOT_PATH = "/Pioneer3DX"
24 CAMERA_PATH = "/Pioneer3DX/LineCamera"
25
26 START_SIMULATION = True
27 SET_ROBOT_POSE_FROM_SCRIPT = True
28 USE_STEPPING = False
29
30 IMG_W = 640
31 IMG_H = 480
32 ORTHO_VIEW_WIDTH_M = 0.187
33
34 LINE_CENTER_WORLD = np.array([0.0, 0.0], dtype=float)
35 LINE_LENGTH_M = 5.0
36 LINE_YAW_RAD = math.radians(0.0)
37
38 ROBOT_Z = 0.138
39 SETTLE_STEPS = 5
40 SETTLE_TIME_SEC = 0.10
41
42 CANNY_LOW = 50
43 CANNY_HIGH = 150
44 HOUGH_THRESHOLD = 50
45 HOUGH_MIN_LINE_LENGTH = 60
46 HOUGH_MAX_LINE_GAP = 20
47
48 DARK_THRESHOLD = 90
49 MIN_DETECTED_POINTS = 20
50
51 TESTS = [
52     ("dist_0_00_angle_0", 0.00, 0.0, 0.0),
53     ("dist_0_02_angle_0", 0.02, 0.0, 0.0),
54     ("dist_0_04_angle_0", 0.04, 0.0, 0.0),
55     ("dist_0_06_angle_0", 0.06, 0.0, 0.0),
56     ("dist_0_08_angle_0", 0.08, 0.0, 0.0),
57
58     ("dist_0_05_angle_p10", 0.05, 10.0, 0.0),
59     ("dist_0_05_angle_m10", 0.05, -10.0, 0.0),
60     ("dist_0_05_angle_p20", 0.05, 20.0, 0.0),
61     ("dist_0_05_angle_m20", 0.05, -20.0, 0.0),
62 ]
63
64 @dataclass
65 class LineEstimate:
66     detected: bool
67     d_m: float = float("nan")
68     phi_rad: float = float("nan")
69     points_count: int = 0
70     method: str = "none"
71
72
73 def ensure_dirs() -> None:
74     os.makedirs(RAW_DIR, exist_ok=True)
75     os.makedirs(PROC_DIR, exist_ok=True)
76     os.makedirs(RESULTS_DIR, exist_ok=True)
77
78

```

```

79 def wrap_to_pi(angle: float) -> float:
80     return (angle + math.pi) % (2.0 * math.pi) - math.pi
81
82
83 def wrap_to_half_pi(angle: float) -> float:
84     angle = wrap_to_pi(angle)
85     if angle > math.pi / 2.0:
86         angle -= math.pi
87     if angle < -math.pi / 2.0:
88         angle += math.pi
89     return angle
90
91 def line_angle_error_deg(phi_true_rad, phi_meas_rad):
92     err = math.degrees(phi_meas_rad - phi_true_rad)
93     err = (err + 90.0) % 180.0 - 90.0
94     return err
95
96 def line_endpoints_world() -> Tuple[np.ndarray, np.ndarray]:
97     direction = np.array([math.cos(LINE_YAW_RAD), math.sin(LINE_YAW_RAD)], dtype
98                           =float)
99     p1 = LINE_CENTER_WORLD - 0.5 * LINE_LENGTH_M * direction
100    p2 = LINE_CENTER_WORLD + 0.5 * LINE_LENGTH_M * direction
101    return p1, p2
102
103 def desired_robot_pose(distance_m: float, rel_angle_deg: float, along_m: float)
104     -> Tuple[float, float, float]:
105     direction = np.array([math.cos(LINE_YAW_RAD), math.sin(LINE_YAW_RAD)], dtype
106                           =float)
107     normal = np.array([-math.sin(LINE_YAW_RAD), math.cos(LINE_YAW_RAD)], dtype=
108                      float)
109     pos = LINE_CENTER_WORLD + along_m * direction + distance_m * normal
110     yaw = LINE_YAW_RAD + math.radians(rel_angle_deg)
111     return float(pos[0]), float(pos[1]), float(yaw)
112
113 def world_to_camera_plane(point_world: np.ndarray, robot_x: float, robot_y:
114     float, robot_yaw: float) -> np.ndarray:
115     c = math.cos(robot_yaw)
116     s = math.sin(robot_yaw)
117     dx = float(point_world[0]) - robot_x
118     dy = float(point_world[1]) - robot_y
119     x_robot_forward = c * dx + s * dy
120     y_robot_left = -s * dx + c * dy
121     x_camera_right = -y_robot_left
122     y_camera_forward = x_robot_forward
123     return np.array([x_camera_right, y_camera_forward], dtype=float)
124
125 def fit_line_params(points_xy: np.ndarray) -> Tuple[float, float, float, float,
126     float]:
127     pts = points_xy.astype(np.float32)
128     vx, vy, x0, y0 = cv2.fitLine(pts, cv2.DIST_L2, 0, 0.01, 0.01).reshape(-1)
129     phi = math.atan2(float(vy), float(vx))
130     phi = wrap_to_half_pi(phi)
131     a = -float(vy)
132     b = float(vx)
133     n = math.sqrt(a * a + b * b)
134     a /= n
135     b /= n

```

```

133     c = -(a * float(x0) + b * float(y0))
134     d = c
135     return d, phi, a, b, c
136
137
138 def true_line_in_camera(robot_x: float, robot_y: float, robot_yaw: float) ->
    Tuple[float, float]:
139     p1_w, p2_w = line_endpoints_world()
140     p1_c = world_to_camera_plane(p1_w, robot_x, robot_y, robot_yaw)
141     p2_c = world_to_camera_plane(p2_w, robot_x, robot_y, robot_yaw)
142     points = np.vstack([p1_c, p2_c])
143     d, phi, _, _, _ = fit_line_params(points)
144     return abs(d), phi
145
146
147 def image_to_metric_points(pixel_points_uv: np.ndarray, width: int, height: int)
    -> np.ndarray:
148     mpp = ORTHO_VIEW_WIDTH_M / float(width)
149     ortho_height_m = ORTHO_VIEW_WIDTH_M * float(height) / float(width)
150     u = pixel_points_uv[:, 0].astype(float)
151     v = pixel_points_uv[:, 1].astype(float)
152     x = (u - 0.5 * (width - 1)) * mpp
153     y = (0.5 * (height - 1) - v) * (ortho_height_m / float(height))
154     return np.column_stack([x, y])
155
156
157 def detect_line_from_image(img_bgr: np.ndarray, debug_path: Optional[str] = None
    ) -> LineEstimate:
158     height, width = img_bgr.shape[:2]
159     gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
160     blur = cv2.GaussianBlur(gray, (5, 5), 0)
161     edges = cv2.Canny(blur, CANNY_LOW, CANNY_HIGH)
162     lines = cv2.HoughLinesP(
163         edges,
164         1,
165         np.pi / 180.0,
166         threshold=HOUGH_THRESHOLD,
167         minLineLength=HOUGH_MIN_LINE_LENGTH,
168         maxLineGap=HOUGH_MAX_LINE_GAP,
169     )
170
171     pts_uv: List[Tuple[int, int]] = []
172     method = "hough"
173     if lines is not None:
174         for item in lines.reshape(-1, 4):
175             x1, y1, x2, y2 = map(int, item)
176             pts_uv.append((x1, y1))
177             pts_uv.append((x2, y2))
178
179     if len(pts_uv) < MIN_DETECTED_POINTS:
180         method = "dark_mask"
181         mask = gray < DARK_THRESHOLD
182         ys, xs = np.where(mask)
183         if len(xs) > 0:
184             step = max(1, len(xs) // 3000)
185             pts_uv = list(zip(xs[::step].astype(int), ys[::step].astype(int)))
186
187     if len(pts_uv) < 2:
188         if debug_path is not None:
189             cv2.imwrite(debug_path, img_bgr)

```

```

190         return LineEstimate(False, points_count=len(pts_uv), method="none")
191
192     points_uv = np.array(pts_uv, dtype=np.float32)
193     points_xy = image_to_metric_points(points_uv, width, height)
194     d, phi, a, b, c = fit_line_params(points_xy)
195
196     if debug_path is not None:
197         debug = img_bgr.copy()
198         if lines is not None:
199             for item in lines.reshape(-1, 4):
200                 x1, y1, x2, y2 = map(int, item)
201                 cv2.line(debug, (x1, y1), (x2, y2), (0, 0, 255), 2)
202             x0_px = int(np.mean(points_uv[:, 0]))
203             y0_px = int(np.mean(points_uv[:, 1]))
204             length = max(width, height)
205             vx_px = math.cos(phi)
206             vy_px = -math.sin(phi)
207             p_a = (int(x0_px - length * vx_px), int(y0_px - length * vy_px))
208             p_b = (int(x0_px + length * vx_px), int(y0_px + length * vy_px))
209             cv2.line(debug, p_a, p_b, (0, 255, 0), 2)
210             cv2.circle(debug, (width // 2, height // 2), 5, (255, 0, 0), -1)
211             cv2.imwrite(debug_path, debug)
212
213     return LineEstimate(True, abs(d), phi, len(pts_uv), method)
214
215
216 def connect_to_coppelia():
217     if RemoteAPIClient is None:
218         raise RuntimeError(
219             "Не найден пакет coppeliasim_zmqremoteapi_client. Установите его : "
220             "pip install coppeliasim-zmqremoteapi-client"
221         )
222     client = RemoteAPIClient()
223     if hasattr(client, "require"):
224         sim = client.require("sim")
225     else:
226         sim = client.getObject("sim")
227     return client, sim
228
229
230 def get_handle(sim, path: str):
231     try:
232         return sim.getObject(path)
233     except Exception as exc:
234         raise RuntimeError(
235             f"Не найден объект {path}. Проверьте имя /alias объекта в сцене "
236             "CoppeliaSim."
237         ) from exc
238
239
240 def get_camera_image_bgr(sim, cam) -> np.ndarray:
241     data = sim.getVisionSensorImg(cam)
242     if isinstance(data, tuple) and len(data) == 2:
243         img, res = data
244     elif isinstance(data, list) and len(data) == 2:
245         img, res = data[0], data[1]
246     else:
247         raise RuntimeError("Неожиданный формат ответа sim.getVisionSensorImg")
248     width, height = int(res[0]), int(res[1])

```

```

249     if isinstance(img, bytes):
250         arr = np.frombuffer(img, dtype=np.uint8)
251     else:
252         arr = np.array(img, dtype=np.uint8)
253     arr = arr.reshape((height, width, 3))
254     arr = cv2.flip(arr, 0)
255     img_bgr = cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
256     return img_bgr
257
258
259 def wait_for_scene_update(client, sim) -> None:
260     if USE_STEPPING:
261         for _ in range(SETTLE_STEPS):
262             client.step()
263     else:
264         for _ in range(SETTLE_STEPS):
265             sim.wait(SETTLE_TIME_SEC)
266
267
268 def save_csv(rows: List[dict]) -> str:
269     path = os.path.join(RESULTS_DIR, "measurements.csv")
270     fieldnames = [
271         "test",
272         "robot_x_m",
273         "robot_y_m",
274         "robot_yaw_deg",
275         "true_distance_m",
276         "measured_distance_m",
277         "distance_error_m",
278         "true_angle_deg",
279         "measured_angle_deg",
280         "angle_error_deg",
281         "detected",
282         "points_count",
283         "method",
284     ]
285     with open(path, "w", newline="", encoding="utf-8") as f:
286         writer = csv.DictWriter(f, fieldnames=fieldnames, delimiter=";")
287         writer.writeheader()
288         writer.writerows(rows)
289     return path
290
291
292 def plot_results(rows: List[dict]) -> None:
293     idx = np.arange(1, len(rows) + 1)
294     detected = np.array([bool(r["detected"]) for r in rows], dtype=bool)
295     dist_err = np.array([float(r["distance_error_m"]) for r in rows], dtype=
float)
296     ang_err = np.array([float(r["angle_error_deg"]) for r in rows], dtype=float)
297     true_d = np.array([float(r["true_distance_m"]) for r in rows], dtype=float)
298     meas_d = np.array([float(r["measured_distance_m"]) for r in rows], dtype=
float)
299     true_phi = np.array([float(r["true_angle_deg"]) for r in rows], dtype=float)
300     meas_phi = np.array([float(r["measured_angle_deg"]) for r in rows], dtype=
float)
301
302     plt.figure(figsize=(9, 5))
303     plt.plot(idx, true_d, marker="o", label="Истинное расстояние")
304     plt.plot(idx, meas_d, marker="s", label="Измеренное расстояние")
305     plt.xlabel("Номер опыта")

```

```

306 plt.ylabel("Расстояние, м")
307 plt.grid(True)
308 plt.legend()
309 plt.tight_layout()
310 plt.savefig(os.path.join(RESULTS_DIR, "distance_true_vs_measured.png"), dpi
    =200)
311 plt.close()
312
313 plt.figure(figsize=(9, 5))
314 plt.plot(idx, dist_err * 100.0, marker="o")
315 plt.xlabel("Номер опыта")
316 plt.ylabel("Ошибка расстояния, см")
317 plt.grid(True)
318 plt.tight_layout()
319 plt.savefig(os.path.join(RESULTS_DIR, "distance_error.png"), dpi=200)
320 plt.close()
321
322 plt.figure(figsize=(9, 5))
323 plt.plot(idx, true_phi, marker="o", label="Истинный угол")
324 plt.plot(idx, meas_phi, marker="s", label="Измеренный угол")
325 plt.xlabel("Номер опыта")
326 plt.ylabel("Угол линии, град")
327 plt.grid(True)
328 plt.legend()
329 plt.tight_layout()
330 plt.savefig(os.path.join(RESULTS_DIR, "angle_true_vs_measured.png"), dpi
    =200)
331 plt.close()
332
333 plt.figure(figsize=(9, 5))
334 plt.plot(idx, ang_err, marker="o")
335 plt.xlabel("Номер опыта")
336 plt.ylabel("Ошибка угла, град")
337 plt.grid(True)
338 plt.tight_layout()
339 plt.savefig(os.path.join(RESULTS_DIR, "angle_error.png"), dpi=200)
340 plt.close()
341
342 plt.figure(figsize=(9, 5))
343 plt.bar(idx, detected.astype(int))
344 plt.xlabel("Номер опыта")
345 plt.ylabel("Детекция: 1 —успешно , 0 —нет ")
346 plt.ylim(-0.05, 1.05)
347 plt.grid(True, axis="y")
348 plt.tight_layout()
349 plt.savefig(os.path.join(RESULTS_DIR, "detection_success.png"), dpi=200)
350 plt.close()
351
352
353 def run_experiment() -> None:
354     ensure_dirs()
355     client, sim = connect_to_coppelia()
356     robot = get_handle(sim, ROBOT_PATH)
357     cam = get_handle(sim, CAMERA_PATH)
358
359     if USE_STEPPING:
360         client.setStepping(True)
361
362     if START_SIMULATION:
363         state = sim.getSimulationState()

```

```

364         if state == sim.simulation_stopped:
365             sim.startSimulation()
366             time.sleep(0.5)
367
368     rows: List[dict] = []
369     for i, (name, distance_m, rel_angle_deg, along_m) in enumerate(TESTS, start
370 =1):
371         robot_x, robot_y, robot_yaw = desired_robot_pose(distance_m,
372 rel_angle_deg, along_m)
373         if SET_ROBOT_POSE_FROM_SCRIPT:
374             sim.setObjectPosition(robot, -1, [robot_x, robot_y, ROBOT_Z])
375             sim.setObjectOrientation(robot, -1, [0.0, 0.0, robot_yaw])
376             wait_for_scene_update(client, sim)
377
378         img_bgr = get_camera_image_bgr(sim, cam)
379         raw_path = os.path.join(RAW_DIR, f"{i:02d}_{name}.png")
380         debug_path = os.path.join(PROC_DIR, f"{i:02d}_{name}_detected.png")
381         cv2.imwrite(raw_path, img_bgr)
382
383         estimate = detect_line_from_image(img_bgr, debug_path=debug_path)
384         true_d, true_phi = true_line_in_camera(robot_x, robot_y, robot_yaw)
385
386         if estimate.detected:
387             measured_d = estimate.d_m
388             measured_phi = estimate.phi_rad
389             d_err = measured_d - true_d
390             phi_err = line_angle_error_deg(true_phi, measured_phi)
391         else:
392             measured_d = float("nan")
393             measured_phi = float("nan")
394             d_err = float("nan")
395             phi_err = float("nan")
396
397         row = {
398             "test": name,
399             "robot_x_m": f"{robot_x:.4f}",
400             "robot_y_m": f"{robot_y:.4f}",
401             "robot_yaw_deg": f"{math.degrees(robot_yaw):.4f}",
402             "true_distance_m": f"{true_d:.5f}",
403             "measured_distance_m": f"{measured_d:.5f}",
404             "distance_error_m": f"{d_err:.5f}",
405             "true_angle_deg": f"{math.degrees(true_phi):.5f}",
406             "measured_angle_deg": f"{math.degrees(measured_phi):.5f}",
407             "angle_error_deg": f"{phi_err:.5f}",
408             "detected": estimate.detected,
409             "points_count": estimate.points_count,
410             "method": estimate.method,
411         }
412         rows.append(row)
413         print(
414             f"[{i:02d}] {name}: detected={estimate.detected}, "
415             f"d_true={true_d:.3f} m, d_meas={measured_d:.3f} m, "
416             f"phi_true={math.degrees(true_phi):.2f} deg, phi_meas={math.degrees(
417                 measured_phi):.2f} deg"
418         )
419
420     csv_path = save_csv(rows)
421     plot_results(rows)
422
423     success_rate = sum(1 for r in rows if r["detected"]) / len(rows)

```

```
421     print("\Готово")
422     print(f"CSV: {csv_path}")
423     print(f"Сырые изображения: {RAW_DIR}")
424     print(f"Изображения с детекцией : {PROC_DIR}")
425     print(f"Графики: {RESULTS_DIR}")
426     print(f"Доля успешных детекций : {success_rate:.3f}")
427
428
429 if __name__ == "__main__":
430     run_experiment()
```